Session 9: Software Architecture Design and Design Patterns

Prepared by –

Name	Roll Number
Aastha Sheoran	2400290120004
Ananya Rajput	2400290120042
Aashi Chaudhary	2400290120003
Aeishikka Tyagi	2400290120023

1. Architecture Design Concepts

Concept	Meaning	In MCMS
Layered	Horizontal layers; each layer uses only the one below it.	Organises the system into Presentation, Application, Business Logic, Data Access and Database layers.
Client-Server	A client sends requests; a server processes them and responds.	React.js client ↔ Express.js/Node.js server over REST. This is the SRS architecture.
Microservices (conceptual)	Small, independently deployable services, each owning its data.	A possible future split of MCMS; not part of the current design.

MCMS combines the first two: a client-server system whose server is internally layered.

2. Layered Architecture of MCMS

- **Presentation Layer** (React.js, CSS / Tailwind CSS, Recharts, Leaflet). *Responsibility:* display data and capture input, with no business rules. *Components:* role-specific dashboards, complaint submission form (category, description, map location, image upload), tracking page, administrative dashboard. *Talks to:* Application Layer via REST/JSON with a JWT.
- **Application / Controller Layer** (Node.js, Express.js, JWT, Multer). *Responsibility:* receive requests, authenticate, authorise, validate and route to services. *Components:* routes, controllers, JWT and role-based middleware, upload and validation middleware. *Talks to:* calls the Business Logic Layer only.
- **Business Logic / Service Layer** (Node.js, bcrypt). *Responsibility:* MCMS rules such as complaint ID generation, verification/rejection, assignment, priority and deadline, status transitions, escalation. *Components:* User, Complaint, Categorization, Assignment, Priority and Deadline, Status Tracking, Evidence, Notification, Feedback, Reporting and AI Assistance modules. *Talks to:* Data Access Layer and external services.
- **Data Access Layer** (Node.js with a MongoDB driver/ODM). *Responsibility:* isolate all database operations from business rules. *Components:* models/schemas, repositories, CRUD operations, data validation. *Talks to:* MongoDB.
- **Database Layer** (MongoDB). *Responsibility:* persistent storage. *Components:* Users, Complaints, Departments, Categories, FieldWorkers, Assignments, Evidence, StatusHistory, Notifications and Feedback collections.

Requests flow downward through the layers and responses return upward; no layer is skipped.

3. Client-Server Architecture

- **Client:** React.js single-page app in the browser; renders role-specific screens and stores the JWT.
- **Server:** Node.js runtime with Express.js; exposes REST endpoints, runs business logic, and talks to MongoDB and external services.
- **REST API:** JSON over HTTPS, e.g. `POST /api/complaints`, `GET /api/complaints/:id`, `POST /api/assignments`, `PATCH /api/complaints/:id/status`, `POST /api/evidence` (endpoints are proposed).
- **Data storage:** MongoDB stores documents; images go to Cloudinary and only their URLs are stored in the Evidence collection.
- **JWT and RBAC:** login compares the password with its bcrypt hash and issues a JWT carrying user ID and role; the client sends it in the `Authorization: Bearer` header; middleware verifies the token, then checks the role (Citizen, Municipal Officer, Field Worker, Administrator) against the endpoint. Citizens see only their own complaints and Field Workers only their assigned tasks.

Example: citizen submits a complaint.

1. The Citizen fills the React form (category, description, map pin, photos) and React sends `POST /api/complaints` with the JWT.
2. Express middleware checks the JWT and Citizen role, handles files with Multer, and validates input.
3. The Complaint service generates the Complaint ID, sets status *Submitted*, and may request AI suggestions (advisory only).
4. The Evidence service uploads photos to Cloudinary; repositories save the Complaint, Evidence and first StatusHistory in MongoDB.
5. The Notification module alerts the citizen and officers; the API returns `201 Created` with the Complaint ID.

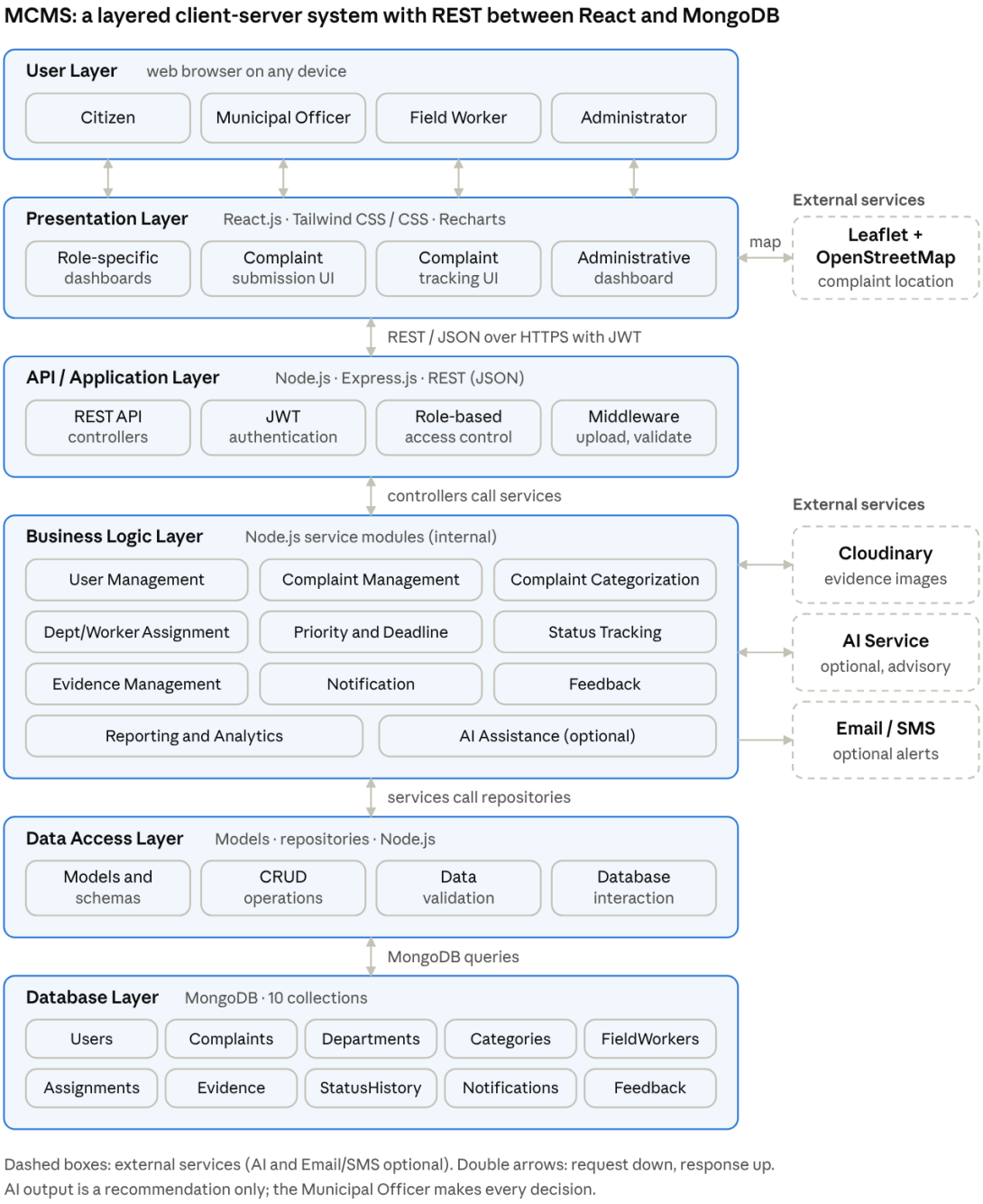
4. Conceptual Microservices Architecture

> *MCMS is **not** implemented as microservices. This is a conceptual future possibility only.*

- **Authentication Service:** registration, login, bcrypt hashing, JWT issuing, roles.
- **Complaint Management Service:** complaint creation, ID generation, categorization, verification/rejection, status history, feedback.
- **Assignment Service:** departments, field-worker assignment, priority, deadlines, overdue escalation.
- **Notification Service:** in-app and optional email/SMS alerts triggered by events.
- **Evidence Management Service:** upload validation, Cloudinary storage, evidence metadata.
- **AI Assistance Service:** classification, priority prediction and duplicate detection as recommendations only.
- **Reporting Service:** aggregated statistics for dashboards and reports.

Services would be reached through an API Gateway and communicate via REST calls and events (e.g. "ComplaintAssigned"), each owning its own data.

5. System Architecture Diagram



Dashed boxes: external services (AI and Email/SMS optional). Double arrows: request down, response up.
AI output is a recommendation only; the Municipal Officer makes every decision.

MCMS system architecture · 6 layers, external services, 3 key flows

Read the diagram top to bottom: users reach the React client, which calls the Express REST API; controllers call the business modules, which use repositories to reach MongoDB, with Cloudinary, maps, optional AI and optional email/SMS at the side.

6. Design Patterns

These patterns are **proposed** for MCMS; none is claimed as already implemented.

Pattern	Purpose	MCMS Example
MVC	Separates data, UI and request handling.	Model: Complaint schema; View: React components; Controller: `ComplaintController` for `POST /api/complaints`.
Repository	Hides database queries behind an interface.	`ComplaintRepository` (`create()`, `findById()`, `findByStatus()`); services never call MongoDB directly.
Service Layer	Keeps business rules in reusable services.	`AssignmentService.assignComplaint()` validates the worker, saves the Assignment, updates status and triggers a notification.
Middleware	Chains request-processing steps in Express.	`authenticateJWT → authorizeRoles('Officer') → validate → controller`.
Observer / Event-Driven	Lets components react to events without tight coupling.	"ComplaintStatusChanged" event observed by the Notification module and the StatusHistory logger.

7. Architecture Comparison and Justification

	Layered	Client-Server	Microservices
Structure	Stacked layers, one-way dependencies	Separate client and server over a network	Many independent services, own databases
Advantages	Separation of concerns; easy testing and maintenance	Central data and security; simple deployment	Independent scaling; fault isolation
Disadvantages	Some overhead for simple operations	Server can be a bottleneck	Complex deployment, data consistency, DevOps
Fit for MCMS	High: structures the backend	High: SRS architecture	Low now; future option

Why layered client-server suits our project: it matches the SRS stack, can be built, tested (Postman) and demonstrated by a four-member team within the academic timeline, and maps directly to our UML models (use cases to controllers, classes to services and models, collections to the database). Centralised JWT and role-based middleware secure all four roles consistently, and the optional AI

module stays isolated so officers retain final authority. Because modules are already separated, heavy ones such as Notification, Evidence, Reporting and AI could later be extracted into microservices if MCMS needed city-wide scale.